

CD-CureNO: Auditable software for restriction-preserving causal neural operators in composite curing

Hyojin Park^a, Nam-Hyun Yoo^b, Jinhong Yang^{c,*}

^a*Gyeongnam Intelligence Innovation Center (GIIC), Kyungnam University, 7
Gyeongnamdaehak-ro, Masanhappo-gu, Changwon 51767, Republic of Korea*

^b*Computer Engineering, Kyungnam University, Changwon 51767, Republic of Korea*

^c*Department of Medical Information Technology, Inje University, Gimhae 50834,
Republic of Korea*

Abstract

CD-CureNO is an Apache-2.0 Python package for auditable neural-operator studies of thermochemical composite curing. It combines conservative one- and two-dimensional reference solvers, immutable manifests and complete-case splits, joint-field and strictly causal operators, deterministic checkpoint inflation that preserves a homogeneous dimensional-reduction limit, managed execution, and independent postflight verification. The tagged release provides command-line workflows, tests, frozen configurations, compact validation records, and number-to-artifact traceability. Existing P0–P5 validation includes a 512-case two-dimensional benchmark, 27 pre-specified solver checks, deterministic replay, and label-free restriction and future-input perturbation tests. These results establish release functionality and numerical contracts, not confirmatory predictive superiority. The package is intended for researchers who need a reusable solver-to-surrogate workflow with explicit provenance, redistribution boundaries, and claim control.

Keywords: research software, neural operator, composite curing, thermochemical simulation, reproducibility, causal machine learning

*Corresponding author

Email address: jinhong@inje.ac.kr (Jinhong Yang)

1 Code metadata

Table 1: SoftwareX code metadata. The C1–C8 labels and field meanings follow the official SoftwareX original-software template, version 6 (March 2026).

Nr	Code metadata description	Metadata
C1	Current code version	v0.0.2
C2	Permanent link to code/repository used for this code version	https://github.com/Jinhong-Yang/CD-CureNO/tree/v0.0.2
C3	Legal code license	Apache License 2.0 (SPDX: Apache-2.0)
C4	Code versioning system used	Git
C5	Software code languages, tools, and services used	Python 3.10–3.12; PyTorch 2.13.0; NumPy 2.2.6; SciPy 1.15.3; pandas 2.3.3; PyArrow 25.0.0; Matplotlib 3.10.9; pytest 9.1.1
C6	Compilation requirements, operating environments, and dependencies	Python package installed with <code>pip install -e ".[dev]"</code> ; CPU execution is supported for audits and tests; CUDA is optional for model training; dependencies are pinned in <code>pyproject.toml</code> .
C7	Link to developer documentation/manual	https://github.com/Jinhong-Yang/CD-CureNO/blob/v0.0.2/README.md
C8	Support email for questions	jinhong@inje.ac.kr

2 1. Motivation and significance

Thermochemical curing models connect an imposed air-temperature program to the temperature and degree-of-cure fields inside a composite-tool assembly. These fields matter because local thermal overshoot, cure gradients, and incomplete cure can affect process design and final part quality. High-fidelity finite element or finite-volume calculations are useful but costly when cure cycles, heat-transfer coefficients, material parameters, and geometry must be varied. Neural operators offer a function-to-function surrogate rather than a model for one fixed input, and Fourier neural operators provide one influential implementation of that idea [1].

Existing composite-curing work already includes physics-informed neural networks [2], residual neural operators [3], and physics-informed neural operators [4]. General scientific machine-learning packages address complementary layers: DeepXDE and PINA provide reusable physics-informed modeling workflows [5, 6], while NeuralOperator supplies tested PyTorch components for operator learning [7]. CD-CureNO is not a replacement for these general libraries. It is a domain-specific integration and verification layer for composite curing, coupling conservative label generation, legacy-behavior audit, cross-dimensional checkpoint transfer, and evidence-bound reporting. It does not claim novelty for using an FNO, jointly predicting temperature and cure, or adding a physics loss. Its software problem is narrower and practical: how can a research team develop, transfer, evaluate, and report cross-dimensional curing operators without losing the provenance and scientific controls needed to interpret the result?

The motivating legacy ResFNO implementation learns one temporal operator at a fixed through-thickness position and reconstructs a field by stacking up to 51 independently trained models [8, 3]. A public implementation is scientifically valuable, but direct reuse exposes common research-code risks: normalization before splitting, inconsistent code paths, batch-local metric aggregation, incomplete field outputs, and ambiguity about whether a displayed position-by-time image is a jointly coupled spatial operator. CD-CureNO provides a behaviorally faithful legacy path and a separately named corrected path, then moves to a single joint 1+1-D model and a true two-spatial-dimensional target. Here, “1+1-D” denotes a jointly predicted through-thickness–time field, whereas “true 2-D” denotes two spatial coordinates, lateral and through-thickness, evolved over time. The terminology is enforced in code, manifests, and reporting artifacts.

Lower-dimensional pretraining and boundary-to-domain lifting have become important precedents for data-efficient operator learning [9, 10]. CD-CureNO adds software support for a stronger engineering invariant: for laterally homogeneous coefficients, initial conditions, and boundary data with zero lateral flux, the target model should recover the source mapping when initialized. The implementation copies the shared through-thickness and temporal computation, initializes new lateral branches as zero residuals, and independently recreates every target tensor before accepting an inflated checkpoint. A separate causal path uses dilated temporal convolutions and a future-perturbation test, following the broader principle that causal response models must exclude future inputs [11].

50 The intended users are researchers in scientific machine learning, com-
 51 posite manufacturing, and computational heat transfer who need more than
 52 model classes. They need immutable split manifests, conservative refer-
 53 ence solvers, resumable runs, provenance-rich outputs, independent postflight
 54 checks, and a claim boundary that distinguishes validation-only evidence
 55 from confirmatory held-out results. CD-CureNO packages those concerns as
 56 one inspectable workflow.

57 2. Software description

58 2.1. Software architecture

59 CD-CureNO follows a staged artifact flow rather than a notebook-only
 60 workflow. Figure 1 summarizes the five principal layers. Data and prove-
 61 nance enter through immutable manifests. Conservative solvers generate
 62 or validate labels. Model modules implement joint, causal, and dimension-
 63 transfer operators. Training modules provide resume support and single-
 64 writer journals. Evaluation and reporting modules independently reconstruct
 65 metrics, bind numbers to run identifiers, and fail closed when an integrity
 66 contract is not satisfied.

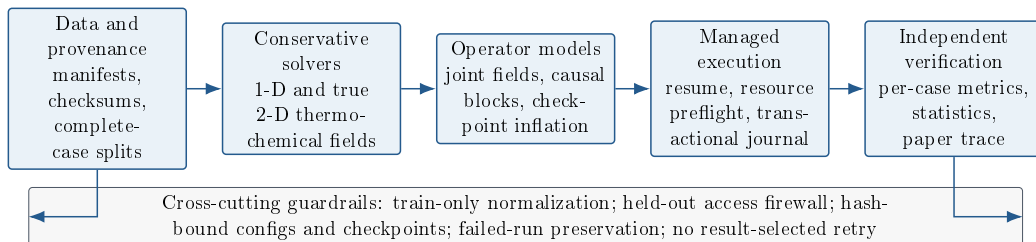


Figure 1: CD-CureNO software architecture. Each stage emits inspectable artifacts consumed by the next stage; cross-cutting guardrails are verified again during independent aggregation and reporting.

67 The package is organized under `src/cdcureno`. The `data` module loads
 68 public and generated arrays, fits normalization on training cases only, and
 69 exposes separate training shards and evaluation readers. The `solvers` mod-
 70 ule implements conservative one- and two-dimensional thermochemical in-
 71 tegration with harmonic face conductance and Robin boundary fluxes. The
 72 `models` module contains joint operators, causal temporal blocks, target oper-
 73 ators, ablations, and deterministic checkpoint-inflation logic. The `training`

74 module records configurations, environment and Git state, histories, check-
75 points, receipts, and process liveness. The **evaluation** and **reporting**
76 modules recompute complete-case metrics from saved predictions and bind
77 manuscript values to exact evidence.

Table 2: Major software functionality and its primary audit artifact.

Function	User-facing capability	Audit output
Legacy audit	Inspect upstream data shapes, splits, normalization, losses, and expected files without modifying the pinned source	JSON/Markdown inventory and regression tests
Reproducible training	Run exact, corrected, joint, and causal experiments with deterministic IDs and resume support	Resolved config, Git/environment snapshot, history, checkpoints, predictions, receipt
Conservative simulation	Generate one- and two-dimensional temperature/cure labels and validate energy closure, refinement, and extrusion	Per-case metadata, convergence tables, immutable manifests
Cross-dimensional transfer	Inflate a one-dimensional source into a target with copied shared tensors and zero-residual lateral adapters	Tensor mapping, tensor hashes, recreated target state, restriction metrics
Causality verification	Perturb all future input channels and compare every saved output on the unperturbed prefix	Cutoff-wise prefix differences and pass/fail contract
Independent reporting	Recompute per-case metrics, release frozen populations, run registered statistics, and trace manuscript values	Postflight reports, release receipts, statistics bundle, number-to-run trace

78 2.2. Typical user workflow and generated artifacts

79 A typical release-level workflow has four auditable checkpoints. First,
80 the user creates an isolated environment, installs the package, and runs the

test suite; tests that depend on separately acquired upstream artifacts are skipped with an explicit reason when those artifacts are absent. Second, the user either audits the pinned ResFNO source or reviews a generated benchmark plan before label generation. Third, a source model can be trained or loaded and inflated into a target model, after which restriction and causality contracts are evaluated independently of predictive labels. Fourth, postflight commands reopen saved predictions, recompute complete-case metrics, and emit a report bound to run, configuration, data, and checkpoint identifiers.

The expected products are not only model weights. Each completed path emits a resolved configuration, environment and Git metadata, immutable data or split manifests, per-case records, checkpoints and predictions where applicable, and a machine-readable verification receipt. This artifact-first interface lets a user inspect or reuse an individual stage without accepting later scientific claims. The tagged release contains the source, tests, frozen configurations, compact validation summaries, and traceability records; data that cannot be redistributed are handled through acquisition or regeneration instructions rather than silently bundled.

2.3. Restriction-preserving and causal target construction

The target model uses canonical input order $[B, N_t, N_z, N_x, C_{\text{in}}]$ and returns temperature and degree of cure over the same time-space grid. For a laterally extruded source prediction, the target is written conceptually as

$$\widehat{T}_{2D}(x, z, t) = \text{Lift}_x \left[\widehat{T}_{1D}(z, t) \right] + \Delta T_x(x, z, t). \quad (1)$$

The new geometry projection and lateral adapter output factors are initialized to zero. Consequently, homogeneous inputs recover the source computation at initialization, up to floating-point ordering effects for $N_x > 1$. The inflation report records copied tensors, initialized tensors, shape changes, and hashes. A verifier then reconstructs the full untrained target from the pinned source checkpoint, configuration, and seed; checking model outputs alone is insufficient because a trained target could accidentally appear lateral-invariant on a small test.

The structurally causal target contains eight dilated temporal convolutions and no temporal FFT modules. Causality is checked by perturbing every input channel after several cutoffs and requiring all temperature, residual, degree-of-cure, and cure-rate prefixes to remain unchanged. Cure is represented through a nonnegative latent rate integrated into a bounded monotone

115 state, while conservative residuals use finite-volume-compatible fluxes rather
116 than global derivatives across a material interface.

117 *2.4. Scientific and execution safeguards*

118 Every expensive run has a resolved configuration, data checksums, ran-
119 dom seed, environment snapshot, checkpoint history, per-case metrics, and
120 physical-unit predictions. Splits are defined by complete cure cycle or com-
121 plete simulation case; spatial and temporal grid points are never treated as
122 independent samples. The package preserves failed and superseded runs, and
123 result indices record only independently verified completions. Managed ex-
124 ecution uses a single-writer lease and journal so interruption cannot silently
125 change the selected checkpoint population.

126 The reporting layer separates structural, validation-only, descriptive, and
127 confirmatory evidence. The current SoftwareX article uses only passed P0–
128 P5 software-validation artifacts. It does not use the unfinished P6 held-out
129 campaign to claim superiority, label savings, state of the art, or broad source-
130 seed invariance.

131 **3. Illustrative examples**

132 *3.1. Installation and audit*

133 The repository targets Python 3.10–3.12. A clean editable installation
134 and CPU test run are:

```
135 python -m venv .venv  
136 # Activate .venv with the command appropriate for the local shell.  
137 python -m pip install --upgrade pip  
138 python -m pip install -e "[dev]"  
139 python -m pytest -q
```

142 The legacy audit is read-only and supports a dry run:

```
143 python scripts/audit_legacy_repo.py \  
144 --repo-root external/ResFNO \  
145 --output-dir outputs/audit \  
146 --dry-run
```

149 A successful audit creates a JSON/Markdown inventory under the re-
150 quested output directory; no upstream file is modified. The implemented
151 CLI also exposes auditable exact, corrected, joint factorized, and joint causal
152 training modes. Independent evaluation reloads a saved run by identifier

153 and recomputes its complete-case metrics. Commands and expected artifact
154 names are documented in the versioned release README.

155 3.2. Conservative two-dimensional benchmark

156 The two-dimensional workflow separates plan generation from label gen-
157 eration, which allows users to review case families and split semantics before
158 solving:

```
159 python -m pytest -q tests/unit/test_p4_dataset_pipeline.py  
160 python scripts/generate_p4_2d.py \  
161     --tier core --plan-only \  
162     --plan tmp/p4_2d_core_v1_plan_release_review.json  
163 python scripts/generate_p4_2d.py --tier core \  
164     --plan splits/p4_2d_core_v1_plan.json  
165 python scripts/validate_p4_solver.py \  
166     --input external/ResFNO/data/Case1.mat  
167 python scripts/verify_p4_gate.py  
168
```

170 The focused tests validate plan integrity and immutable-generation con-
171 tracts. The plan-only command writes a noncanonical release-review copy
172 without field labels; it does not replace the pre-label plan. Production gen-
173 eration loads the included immutable plan and creates the approximately 918
174 MB array set. Solver validation requires the separately acquired, checksum-
175 verified `Case1.mat`; the final gate requires the generated arrays. Storage,
176 worker, resume, and expected-output details are in the release README.

177 The frozen core benchmark contains 512 cases with temperature/cure
178 field shape $[512, 112, 50, 40]$. The production solver passed 27 pre-specified
179 checks; all cases completed, the maximum relative global energy residual
180 was 1.345×10^{-12} , and 11 cases selected from plan metadata reproduced
181 their saved float32 slice hashes exactly. The independent SciPy-BDF check
182 reassembled fluxes and time integration but shared the grid, material values,
183 and cure law, so it is not described as a fully independent external solver.

184 3.3. Checkpoint inflation and restriction verification

185 The command interfaces can be inspected in a clean checkout without
186 the non-redistributed artifacts:

```
187 python scripts/inflate_causal_1d_to_2d_checkpoint.py --help  
188 python scripts/verify_p5_causal_restriction.py --help  
189
```


Full verification requires the P3 source checkpoint, the persisted inflated target, the P3 data/split, and the P4 plan, manifest, ID split, and generated array root. The target must therefore be written before it is verified:

```
python scripts/inflate_causal_1d_to_2d_checkpoint.py \
  --source outputs/runs/p3-source-causal-v1-seed0-fix1/checkpoints/best.
  pt \
  --output outputs/checkpoints/p5_causal_inflated_source.pt
python scripts/verify_p5_causal_restriction.py \
  --source-checkpoint outputs/runs/p3-source-causal-v1-seed0-fix1/
  checkpoints/best.pt \
  --target-checkpoint outputs/checkpoints/p5_causal_inflated_source.pt \
  --source-data data/processed/p3_source_1d_v3.npz \
  --p4-array-root data/processed/p4_2d_core_v1
```

The remaining frozen config and split paths use the documented defaults. Here ‘-dry-run’ suppresses output writes but does not remove any prerequisite.

For the saved causal target, 100 source tensors were copied bitwise and 17 target tensors were initialized; the complete 117-tensor state matched a deterministic reconstruction. On the actual homogeneous P4 validation input, temperature restriction relative L_2 and lateral-invariance scores were 3.33×10^{-8} and 1.76×10^{-8} , respectively. The maximum future-prefix difference was exactly zero across three cutoffs, four output fields, and perturbations to all 20 target channels. These checks use label-free homogeneous inputs and establish initialization behavior and architectural causality, not predictive superiority.

3.4. Joint-field reconstruction

Figure 2 illustrates an output of the earlier joint 1+1-D stage. The case identifier was fixed before plotting, and shared color scales permit visual comparison. The figure is deliberately labelled as a through-thickness-by-time result, not a true two-spatial-dimensional field.

The factorized and causal joint models used 198,282 and 225,162 parameters, respectively, compared with 15,076,467 parameters across 51 independently trained legacy models. Their seed-0 complete-case temperature relative L_2 means were 0.001192 and 0.001310; all saved cure predictions respected bounds and monotonicity, and the causal model showed exactly zero prefix change in its fixed future-perturbation test. These are staged, one-seed validation results and are included only to demonstrate the software’s joint-field and causality workflows.

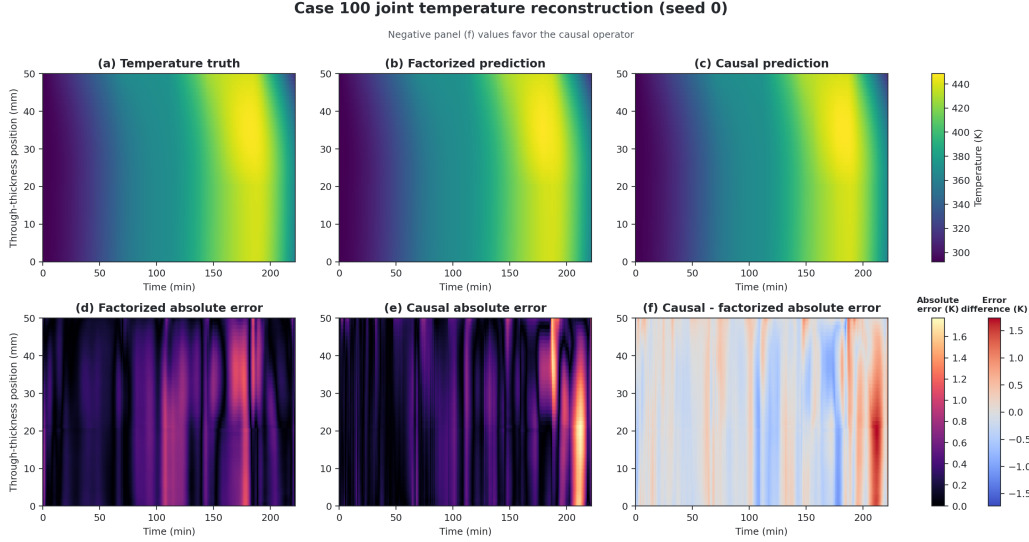


Figure 2: Representative joint 1+1-D temperature reconstruction for fixed test case 100 and seed 0. The displayed configurations were fixed by the validation protocol, and the case identifier was specified before plotting. This single-case display is descriptive and does not establish statistical superiority.

231 4. Impact

232 CD-CureNO’s primary present impact is methodological infrastructure.
 233 First, it makes the behavior of a legacy curing-operator codebase inspectable.
 234 Exact and corrected paths remain separately named, allowing a researcher
 235 to reproduce reported behavior without quietly repairing it, then compare
 236 against a fairer implementation. The audit detected and regression-tested
 237 nine issues, including normalization leakage, an unused smoothness term,
 238 inconsistent function calls, and incomplete field output expectations. This
 239 reduces the risk that later scientific conclusions rest on undocumented im-
 240 plementation differences.

241 Second, the software connects conservative simulation to operator learn-
 242 ing in one repository. The one-dimensional solver is checked against 200
 243 public cases, refinement, and energy closure. The two-dimensional solver is
 244 checked against an extruded one-dimensional limit, a method-of-lines BDF
 245 calculation, energy invariants, deterministic replay, and complete dataset
 246 manifests. The same case identifiers, masks, coordinates, and hashes then
 247 flow into training and evaluation. This shared contract is useful for studying

Table 3: Evidence used to validate the current software release. Performance entries marked descriptive or validation-only are not confirmatory claims.

Stage	Population	Verified result	Claim boundary
P0–P1	Public Case1; 200 cycles, 51 positions, 223 times	Nine legacy defects regression-tested; exact and corrected field workflows independently reconstructed	Reproduction and audit behavior; no corrected-accuracy gain claimed
P2	125 complete test cases; seed 0	Joint factorized rel $L_2 = 0.001192$; causal rel $L_2 = 0.001310$; zero causal prefix change	Descriptive one-seed phase gate; 1+1-D only
P3	200 public solver cases	Mean temperature rel $L_2 = 0.001143$; maximum global energy residual 5.304×10^{-13}	Open-solver agreement; proprietary COM-SOL parity not established
P4	512 generated true-2-D cases	27 solver checks passed; zero failed cases; deterministic replay passed	Public-data-anchored numerical benchmark; no geometry-OOD claim
P5	Label-free restriction tests and one-seed validation pilot	Causal restriction and future-invariance gates passed; transfer pilot passed at budgets 8 and 16	Initialization and functionality evidence; pilot comparison is validation-only

248 transfer without treating generated labels as unquestioned ground truth.

249 Third, CD-CureNO turns restriction preservation and causal response
250 into executable tests. A researcher can inspect which tensors were copied,
251 which were initialized, whether a lateral zero mode can change the source
252 mapping, and whether any future input changes a predicted prefix. These
253 facilities can support new questions about when lower-dimensional pretrain-
254 ing helps, when the benefit disappears as target labels increase, and whether
255 physical invariants improve worst-case behavior. They can also be reused
256 as test patterns for other evolution equations with a known dimensional-

257 reduction limit.

258 Fourth, the managed-run and reporting layers preserve negative evidence.
259 Interrupted sessions, failed development variants, superseded cohorts, and
260 unknown wall-time intervals are retained rather than zero-imputed or re-
261 placed. Manuscript tokens can be resolved only from a verified statistics
262 pointer and an explicit run population. This supports the software-citation
263 principle that a cited version should be identifiable and recoverable [12].

264 The current repository has no defensible public download, user, or com-
265 mercial adoption count, so this article makes no adoption claim. The tagged
266 source, license, versioned README, tests, and validation records are publicly
267 inspectable [13]. CD-CureNO is released under the OSI-approved Apache
268 License 2.0, which permits use, modification, and redistribution, including
269 commercial use, subject to its terms. External ResFNO code and data are
270 not redistributed under an assumed license; the release retains documented
271 acquisition paths and checksums. The generated P4 arrays are likewise not
272 part of the code release, but their deterministic plan, generator configura-
273 tion, canonical manifest hash, and compact validation records are included.
274 This separation makes the reuse boundary explicit rather than presenting
275 unavailable bytes as open data.

276 Scientific scope also remains bounded. The canonical benchmark uses
277 a rectangular grid, reference-temperature conductivity, and a unidirectional
278 material model. Geometry OOD is deferred. The BDF cross-check is in-
279 dependent in flux assembly and integration but shares physical inputs. P2
280 and P5 model comparisons are one-seed or validation-only. The unfinished
281 P6 confirmatory campaign and context-only external sources are not used in
282 this paper as evidence of superiority, broad generalization, or experimental
283 validation.

284 5. Conclusions

285 CD-CureNO packages an integrated P0–P5 workflow for studying cross-
286 dimensional neural operators in composite curing: legacy audit, conservative
287 one- and two-dimensional solvers, immutable data manifests, joint and causal
288 models, deterministic restriction-preserving inflation, resumable managed ex-
289 ecution, independent evaluation, and traceable reporting. The current vali-
290 dation demonstrates release functionality and staged scientific readiness while
291 explicitly preserving the boundary between validation and confirmation. The

292 package is intended to make future claims easier to audit, reproduce, narrow,
293 or reject when the evidence requires it.

294 **Data and code availability**

295 The exact software version described in this article is available from the
296 following repository:

297 `https://github.com/Jinhong-Yang/CD-CureNO/tree/v0.0.2`

298 The open-source release license is:

299 Apache License 2.0 (SPDX: Apache-2.0)

300 The tagged repository contains the installable source, tests, frozen config-
301 urations, compact P0–P5 validation records, and the claim-to-artifact trace-
302 ability table. Two data classes are intentionally not bundled. First, the
303 upstream ResFNO files `Case1.mat` and `double_hold_200_HL.mat` have no
304 license assertion recorded by this project; users must obtain them from the
305 upstream source under its terms and verify the checksums in `DATA_MANIFEST.`
306 `csv`. Second, the generated P4 core arrays (approximately 918 MB) are not
307 distributed in the tagged Git release. The deterministic plan, generator con-
308 figuration, code, canonical manifest hash, and compact gate summaries are
309 included so that users can regenerate and audit the benchmark locally. The
310 Apache-2.0 license applies to CD-CureNO code and does not relicense third-
311 party or generated data.

312 **CRedit author statement**

313 Hyojin Park: Conceptualization, Methodology, Investigation, Data cu-
314 ration, Visualization, Writing – original draft, Writing – review & editing.
315 Nam-Hyun Yoo: Investigation, Formal analysis, Validation, Writing – review
316 & editing. Jinhong Yang: Conceptualization, Supervision, Project adminis-
317 tration, Funding acquisition, Writing – review & editing.

318 **Funding**

319 This work was supported by the Institute of Information & Communi-
320 cations Technology Planning & Evaluation (IITP) – Innovative Human Re-
321 source Development for Local Intellectualization program grant funded by
322 the Korea government (MSIT) (IITP-2026-RS-2024-00436773).

323 Acknowledgements

324 None.

325 Declaration of competing interest

326 The authors declare that they have no known competing financial inter-
327 ests or personal relationships that could have appeared to influence the work
328 reported in this paper.

329 Declaration of generative AI and AI-assisted technologies in the 330 manuscript preparation process

331 During the preparation of this work, the authors used OpenAI ChatGPT
332 (GPT-5.6 Pro) to assist with journal-specific structural review, language edit-
333 ing, and consistency checking. After using this tool, the authors reviewed
334 and edited the content as needed and take full responsibility for the content
335 of the publication.

336 References

- 337 [1] Z. Li, N. Kovachki, K. Azizzadenesheli, B. Liu, K. Bhattacharya, A. Stu-
338 art, A. Anandkumar, Fourier neural operator for parametric partial dif-
339 ferential equations, in: International Conference on Learning Represen-
340 tations, 2021.
341 URL <https://openreview.net/forum?id=c8P9NQVtmn0>
- 342 [2] S. Amini Niaki, E. Haghighat, T. Campbell, A. Poursartip, R. Vaziri,
343 Physics-informed neural network for modelling the thermochemical cur-
344 ing process of composite-tool systems during manufacture, Computer
345 Methods in Applied Mechanics and Engineering 384 (2021) 113959.
346 doi:10.1016/j.cma.2021.113959.
- 347 [3] G. Chen, Y. Li, X. Liu, C. Mehdi-Souzani, Q. Meng, J. Zhou, X. Hao,
348 Physics-guided neural operator for data-driven composites manufactur-
349 ing process modelling, Journal of Manufacturing Systems 70 (2023) 217–
350 229. doi:10.1016/j.jmsy.2023.07.010.

- [4] Q. Meng, Y. Li, X. Liu, G. Chen, X. Hao, A novel physics-informed neural operator for thermochemical curing analysis of carbon-fibre-reinforced thermosetting composites, *Composite Structures* 321 (2023) 117197. doi:10.1016/j.compstruct.2023.117197.
- [5] L. Lu, X. Meng, Z. Mao, G. E. Karniadakis, DeepXDE: A deep learning library for solving differential equations, *SIAM Review* 63 (1) (2021) 208–228. doi:10.1137/19M1274067.
- [6] D. Coscia, A. Ivagnes, N. Demo, G. Rozza, Physics-informed neural networks for advanced modeling, *Journal of Open Source Software* 8 (87) (2023) 5352. doi:10.21105/joss.05352.
- [7] J. Kossaifi, N. Kovachki, Z. Li, D. Pitt, M. Liu-Schiaffini, R. J. George, B. Bonev, K. Azizzadenesheli, J. Berner, A. Anandkumar, A library for learning neural operators, *arXiv preprint arXiv:2412.10354* (2024). doi:10.48550/arXiv.2412.10354.
- [8] G. Chen, Y. Li, X. Liu, Q. Meng, J. Zhou, X. Hao, Residual fourier neural operator for thermochemical curing of composites, *arXiv preprint arXiv:2111.10262* (2021). doi:10.48550/arXiv.2111.10262.
- [9] A. Hemmasian, A. Barati Farimani, Pretraining a neural operator in lower dimensions, *Transactions on Machine Learning Research* (2024). URL <https://openreview.net/forum?id=ZewaRoZehI>
- [10] A. Kashi, A. Daw, M. Gopalakrishnan Meena, H. Lu, Learning the boundary-to-domain mapping using lifting product fourier neural operators for partial differential equations, in: *ICML 2024 AI for Science Workshop*, 2024. arXiv:2406.16740. URL <https://arxiv.org/abs/2406.16740>
- [11] L. Liu, K. Nath, W. Cai, A causality-deeponet for causal responses of linear dynamical systems, *Communications in Computational Physics* 35 (5) (2024) 1194–1228. doi:10.4208/CICP.0A-2023-0078.
- [12] A. M. Smith, D. S. Katz, K. E. Niemeyer, FORCE11 Software Citation Working Group, Software citation principles, *PeerJ Computer Science* 2 (2016) e86. doi:10.7717/peerj-cs.86.

382 [13] H. Park, N.-H. Yoo, J. Yang, CD-CureNO v0.0.2, apache-2.0 licensed
383 software release (2026).
384 URL [https://github.com/Jinhong-Yang/CD-CureNO/releases/](https://github.com/Jinhong-Yang/CD-CureNO/releases/tag/v0.0.2)
385 [tag/v0.0.2](https://github.com/Jinhong-Yang/CD-CureNO/releases/tag/v0.0.2)